



Service Flow

Real-Time Service Request Management System — Technical Documentation

Prepared for the Techno Haven take-home technical assignment. Covers system overview, architecture, database schema, ERD, API design, WebSocket/real-time flow, concurrency model, and end-to-end workflow.

Live demo: kitesrealtimeheavenservicesystem.bolt.host

Repository: github.com/asifms666/Real-Time-Service-Request-Management-System

Submitted by:

Md. Asif mahmud Shakil
+8801608433108

1. Overview

ServiceFlow is a full-stack, real-time service request (IT helpdesk-style) management system. Operators submit service requests; supervisors monitor them live as they move through processing. A background worker automatically picks up each new request and advances it through **pending** → **in_progress** → **completed**, pushing every change to connected clients over WebSocket — no page refresh, no polling.

Business Objectives

- Replace manual request tracking with an automated, real-time system
- Give supervisors live visibility into request progress
- Support multiple requests being processed concurrently
- Stay responsive even while long-running operations execute in the background

Users

Operators — submit new requests with title, description, priority and category.

Supervisors — monitor all requests live, update statuses, cancel requests, view event timelines.

Scope

In scope: Request CRUD, real-time monitoring, concurrent background processing, search/filter, event audit trail, dashboard analytics.

Out of scope: Authentication/authorization, file uploads, email notifications, mobile native apps, multi-tenant isolation.

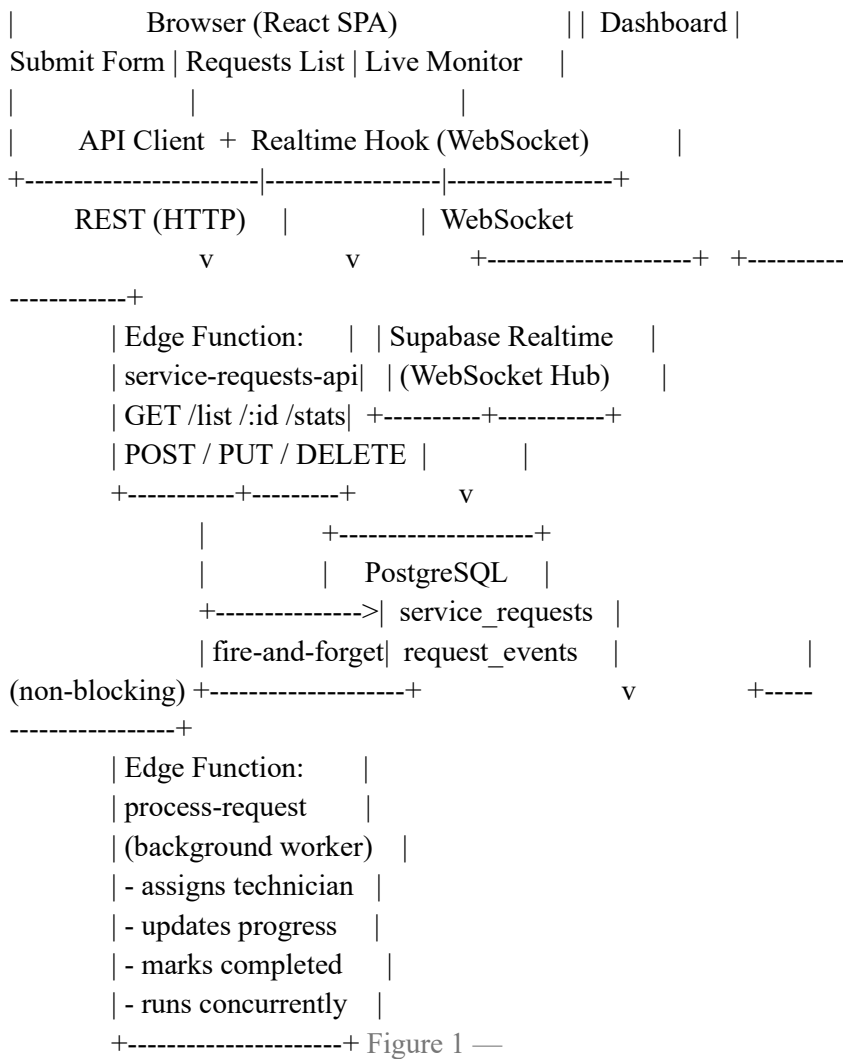
2. Technology Stack

Layer	Technology	Why
Frontend	React 18 + TypeScript + Vite	Type safety, fast dev loop, modern ecosystem
Styling	Tailwind CSS	Utility-first, responsive, consistent design system
Backend (REST API)	Supabase Edge Functions (Deno)	Serverless, auto-scaled, no server to manage
Database	Supabase (PostgreSQL)	Managed Postgres with constraints, triggers, realtime
Real-Time	Supabase Realtime (WebSocket)	Postgres row changes pushed over WebSocket – no polling
Concurrency	Edge Function fire-and-forget	Each request triggers an independent background worker

3. High-Level Architecture

The frontend (React SPA) talks to the backend two ways: REST calls for creating/reading data, and a persistent WebSocket connection for live updates. Every request submitted through the REST API is written to PostgreSQL and immediately triggers a background worker (a separate Edge Function) without blocking the API response.

+-----+



Component Breakdown

Frontend Backend -----

-----src/ App.tsx (routing, realtime sub)

supabase/functions/

components/ service-requests-api/index.ts (REST API) Sidebar,

TopBar, RequestCard, process-request/index.ts (worker)

FilterBar, Badges, ProgressBar, _shared/cors.ts Modal,

Toast, Skeletons

pages/ Database (Postgres) DashboardPage,

SubmitPage, service_requests (main table)

RequestsPage, DetailPage, request_events (audit log)
MonitorPage triggers + RLS policies lib/ types.ts,
supabase.ts, api.ts, realtime.ts, utils.ts

Figure 2 — Frontend / backend component
map

Recommended actual application workflow

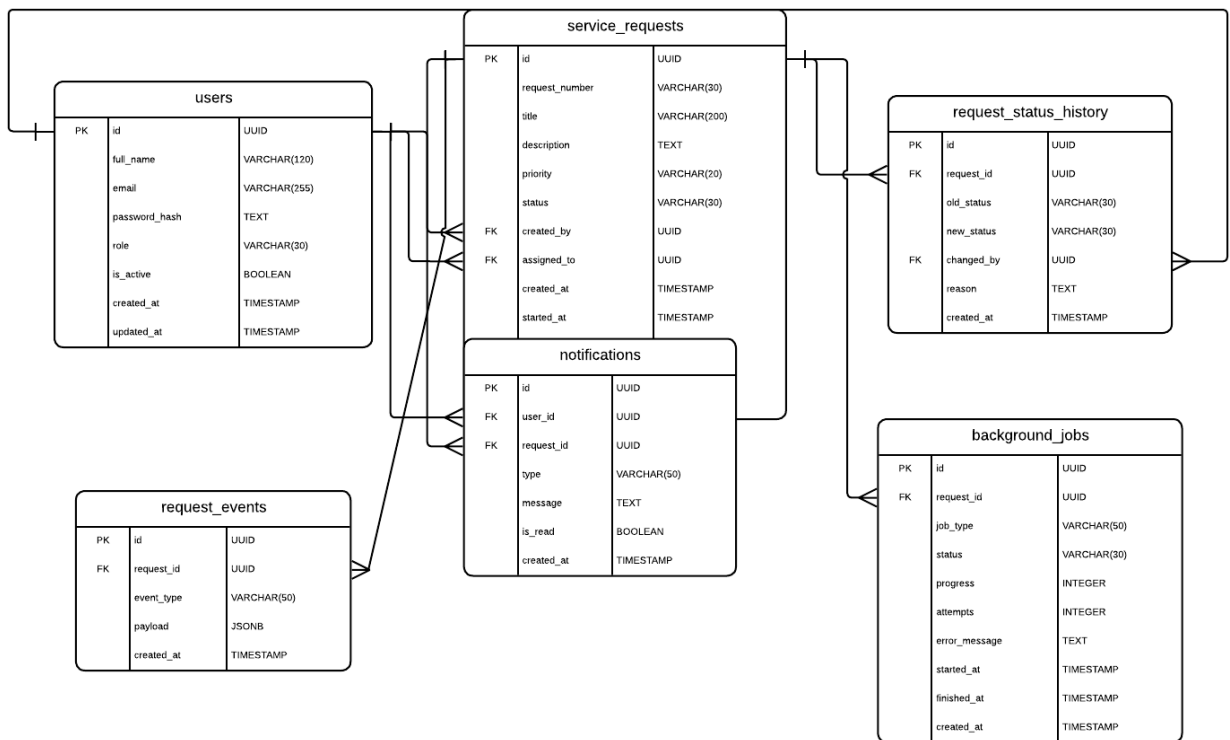
For your implementation, I would structure
the system like this:

Recommended actual application workflow

For your implementation, I would structure the system like this:

4. Database Schema & ERD

Two tables: **service_requests** holds each ticket's current state; **request_events** is an append-only audit log, one row per status/progress change, linked 1-to-many to its parent request.



ServiceFlow — Entity Relationship Diagram

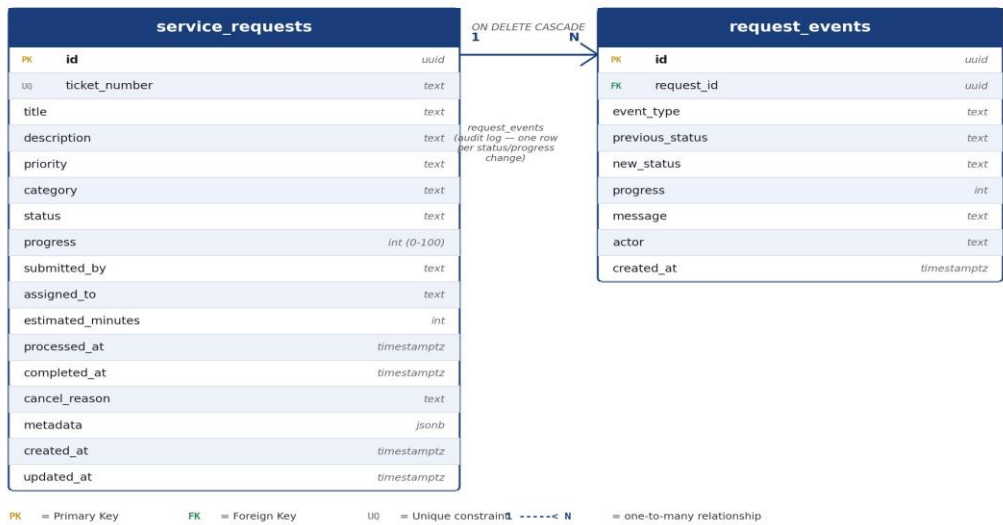


Figure 3 — Entity-Relationship Diagram

Constraints

- priority — CHECK IN ('low','medium','high','critical')
- category — CHECK IN ('hardware','software','network','account','general','other')
- status — CHECK IN ('pending','in_progress','on_hold','completed','cancelled')
- progress — CHECK BETWEEN 0 AND 100

- `estimated_minutes` — CHECK BETWEEN 1 AND 120
- `ticket_number` — UNIQUE
- `request_events.request_id` — FOREIGN KEY -> `service_requests.id` ON DELETE CASCADE

Triggers

- **`update_updated_at`** — auto-sets `updated_at` on every UPDATE.
- **`log_request_change`** — inserts an event row on INSERT, or when status / progress / assignment changes.

Both tables are added to the **supabase_realtime** publication, so every row change is replicated to connected WebSocket clients automatically.

5. API Documentation

Base URL: `{SUPABASE_URL}/functions/v1/service-requests-api`

Auth: Authorization: Bearer `{SUPABASE_ANON_KEY}`

Method	Endpoint	Purpose
GET	/	List requests — search, filter, sort, paginate
GET	/stats	Dashboard stats: total, byStatus, byPriority, avgProgress, last24h
GET	/:id	Get one request including full event timeline
POST	/	Create a request (201) — validates and auto-generates <code>ticket_number</code>
PUT	/:id	Update status, priority, category, assignee, progress, <code>cancel_reason</code> , etc.
DELETE	/:id	Delete a request

POST / body: title (3-200 chars), description (10-2000 chars), priority, category, `submitted_by`, `assigned_to` (optional), `estimated_minutes` (1-120), metadata (optional). Validation failures return 422 with an error + details array.

6. WebSocket / Real-Time Communication

1. Frontend opens a WebSocket connection to Supabase Realtime on app load.
2. It subscribes to `postgres_changes` on `service_requests` and `request_events`.
3. Any INSERT/UPDATE (from the REST API or the background worker) fires through the `realtimepublication`.
4. Supabase Realtime pushes the change to every connected client.
5. The frontend merges the update into local state — the UI updates live, with no polling.

Event	Table	Trigger	Client Action
INSERT	service_requests	New request created	Add to list, show in monitor
UPDATE	service_requests	Status/progress changed	Update card, progress bar, badges
DELETE	service_requests	Request deleted	Remove from list
INSERT	request_events	New event logged	Append to timeline

Connection states shown in the UI: **Connected** (green, live), **Connecting** (amber), **Disconnected** (red, reconnecting).

7. Concurrency Model

1. Operator submits a request via `POST /service-requests-api`.
2. The API inserts the row into PostgreSQL and immediately returns 201 Created.
3. Before returning, the API fires a non-blocking `fetch()` to the `process-request` edge function — it is `notawaited`.
4. `process-request` runs independently: assigns a technician, sets status to `in_progress`, advances progress 5% -> 95% in 10 steps, then marks completed at 100%.
5. Each progress write hits PostgreSQL, which triggers a WebSocket push to all clients.
6. Multiple submitted requests trigger multiple independent worker invocations, running concurrently as separate Deno instances — one slow request never blocks another.

8. Key Assumptions

- Single-tenant demo — no authentication; all users share the same data (RLS allows anon + authenticated access intentionally, for demo purposes).
- Processing is simulated — worker delay is proportional to `estimated_minutes`; production would call `realSERVICE` workflows.
- Technicians are auto-assigned randomly from a pool; production would use load-balancing.
- Ticket numbers are sequential (SR-0001...), generated server-side.
- No file uploads — requests are text-only; attachments are a future enhancement.
- Real-time uses Postgres logical replication via Supabase Realtime — true server push, not polling.